Absolutely — and you're making a smart architectural move. A **Python "reference model" state machine** gives you (1) a deterministic truth source for TDD, (2) a concrete contract for your new Swift UI, and (3) a place to encode Meade/LX200GPS quirks (busy/NAK, multi-step operations, etc.) before you refactor your app.

Below is a **scoping blueprint** you can use as the foundation. It's intentionally structured so you can start small (connection + pointing + slewing) and grow into alignment, GPS, focuser, park/home, etc., while staying testable.

1) What we can ground from the LX200/LX200GPS serial protocol

Key protocol behaviors you'll want to bake into your model early:

- Commands are ASCII and **terminated by `#`** (and many responses are #-terminated strings). [Meade Tele...d Protocol]
- For LX200GPS, the scope may return **NAK (0x15) within ~10ms** if the control chain is busy; the controller should **wait and retry**. [Meade Tele...d Protocol]
- The **alignment / mount mode query** is via **ACK (0x06)** and returns mode codes (AltAz/Land/Polar etc.). [Meade Tele...d Protocol]
- Slewing to a target is typically **`:MS#`** and can return **`0` (ok), `1<string>#` (below horizon), `2<string>#` (above limit)**. [Meade Tele...d Protocol]
- There are explicit commands for:
  - **stop motion** `:Q#` (and directional halts `:Qe#` `:Qn#` `...`) [Meade Tele...d Protocol]
  - **manual directional motion** `:Me#` `:Mn#` `:Ms#` `:Mw#` [Meade Tele...d Protocol]
  - **rate changes** `:RC#` `:RG#` `:RM#` `:RS#` [Meade Tele...d Protocol]
  - **focus** `:F+` `:F-` `:FQ#` and speed settings [Meade Tele...d Protocol]
  - **GPS update** `:gT#` (may take minutes, interrupts normal ops) [Meade Tele...d Protocol]
  - **sleep/wake/park/home** (`:hN#` sleep, `:hW#` wake, `:hP#` park, `:hS#`/`:hF#` home actions, `:h?#` home query) [Meade Tele...d Protocol]
- The spec explicitly warns: some commands may be accepted but **do nothing / return static responses** depending on model/firmware. Your state machine should be resilient to that. [Meade Tele...d Protocol]

That's enough to define a robust model without guessing.

2) Scope the state machine as orthogonal regions (this is the big win)

Instead of one giant "state", split into parallel regions (statechart style). This keeps the model composable and testable:

Region A — Link / Session

Represents serial connection and protocol readiness.

States

- `DISCONNECTED`
- `CONNECTING` (port open + initial handshake)
- `CONNECTED` (ready to send commands)
- `RECOVERING` (after timeouts / repeated NAK / parse errors)
- `FAULTED` (non-recoverable until reset)

Core events

- `open_port`, `close_port`
- `rx_bytes`, `tx_command`
- `timeout`
- `nak_busy` (0x15) [Meade Tele...d Protocol]
- `handshake_ok` (e.g., ACK query returns a mode) [Meade Tele...d Protocol]

Design rule

- The link region owns the *command pipeline*: retry/backoff on NAK, timeouts, "in-flight" request tracking.

Region B — Mount Operating Mode

Represents alignment/mount mode and "high precision" toggle.

States / properties

- `mount_mode ∈ {ALT_AZ, POLAR, LAND, UNKNOWN}`
  - derived from ACK query (0x06) [Meade Tele...d Protocol]
- `precision ∈ {LOW, HIGH}` toggled via `:P#` (returns "HIGH PRECISION" / "LOW PRECISION") [Meade Tele...d Protocol]
- `tracking_rate` (query `:GT#`, set `:STTT.T#`) [Meade Tele...d Protocol]

This region is mostly *data + small transitions*, not a big state tree.

Region C — Motion / Activity

This is the heart of user-visible behavior.

States

- `IDLE_TRACKING`
- `SLEWING_TO_TARGET`
- `MANUAL_MOVING` (directional)
- `PARKING`
- `HOMING`
- `ALIGNING` (if you model `:Aa#` etc.)
- `SLEEPING`

Transitions (examples grounded in protocol)

- `IDLE_TRACKING --(cmd :MS# accepted)--> SLEWING_TO_TARGET` [Meade Tele...d Protocol]
- `SLEWING_TO_TARGET --(cmd :Q#)--> IDLE_TRACKING` [Meade Tele...d Protocol]
- `IDLE_TRACKING --(cmd :Me#/:Mn#/:Ms#/:Mw#)--> MANUAL_MOVING` [Meade Tele...d Protocol]
- `MANUAL_MOVING --(cmd :Qe#/:Qn#/:Qs#/:Qw# or :Q#)--> IDLE_TRACKING` [Meade Tele...d Protocol]
- `ANY --(cmd :hP#)--> PARKING` [Meade Tele...d Protocol]
- `ANY --(cmd :hN#)--> SLEEPING` and `SLEEPING --(cmd :hW#)--> IDLE_TRACKING` [Meade Tele...d Protocol]

Error branches

- `:MS#` returns `1<string>#` (below horizon) or `2<string>#` (above limit) → transition to `IDLE_TRACKING` with a recorded `last_error`. [Meade Tele...d Protocol]

Practical note: For LX200GPS you can also query a "slew status-ish" indicator with `:D#` (bars during slew, null when complete in some implementations). This is useful to drive completion in your simulation. [Meade Tele...d Protocol]

Region D — Pointing Model (Current vs Target coordinates)

Keep this as data + invariants. You don't need full physics to start.

State fields

- `current_ra`, `current_dec` (query `:GR#` and `:GD#`) [Meade Tele...d Protocol]
- `current_alt`, `current_az` (query `:GA#`, `:GZ#`) [Meade Tele...d Protocol]
- `target_ra`, `target_dec` (set `:Sr...#`, `:Sd...#`) [Meade Tele...d Protocol]
- `target_alt`, `target_az` (set `:Sa...#`, `:Sz...#` where supported; altitude set is shown) [Meade Tele...d Protocol]

Invariants to enforce (great for TDD)

- If `motion == SLEWING_TO_TARGET`, then `target_*` must be non-null.
- If `motion == MANUAL_MOVING`, no `:MS#` should be "in-flight" at the same time (serialize).

Region E — Accessories (Focuser / GPS / Misc)

Model as independent subregions so they don't explode your motion state.

Focuser subregion

- `FOCUS_IDLE`, `FOCUS_IN`, `FOCUS_OUT`
- events: `:F+`, `:F-`, `:FQ#` [Meade Tele...d Protocol]

GPS subregion

- `GPS_OFF`, `GPS_ON`, `GPS_UPDATING`
- events: `:g+`, `:g-`, `:gT#` [Meade Tele...d Protocol]

Important: `:gT#` "interrupts normal handbox operations" and can take minutes — in your reference model, treat it as a long-running operation that *blocks* other commands unless you explicitly decide to queue them. [Meade Tele...d Protocol]

3) Define the command contract: inputs, outputs, and "truth updates"

To be useful for TDD, your model should make an explicit promise:

Inputs to the model

1. **App intents** (high-level): `connect`, `goto_ra_dec`, `stop`, `move(direction)`, `park`, `sleep`, `set_site`, etc.
2. **Raw protocol** (optional mode): actual `tx` and `rx` bytes to validate parsing behavior.

Outputs from the model

- Updated `TelescopeState` (a pure data snapshot)
- A list of **protocol actions** to send (e.g., `:Sd...#`, `:Sr...#`, `:MS#`)
- A list of **domain events** for the UI (`SLEW_STARTED`, `SLEW_REJECTED_BELOW_HORIZON`, `NAK_RETRY`, etc.)

This "pure core + effects" split is the key to making it deterministic and testable.

4) Minimal "Phase 1" scope (start here, it's enough for your Swift UI refactor)

If you want the smallest state machine that still pays off immediately:

Implement in Phase 1

- Link/session: connect, handshake via ACK query, NAK retry/backoff [Meade Tele...d Protocol]
- Coordinate query/update:
  - `:GR#`/`:GD#` current position [Meade Tele...d Protocol]
  - `:Sr`/`:Sd` set target [Meade Tele...d Protocol]
- Motion:
  - `:MS#` goto target + handle return codes 0/1/2 [Meade Tele...d Protocol]
  - `:Q#` stop [Meade Tele...d Protocol]
  - Manual moves `:Me`/`:Mn`/`:Ms`/`:Mw` [Meade Tele...d Protocol]
- Rate selection `:RC`/`:RG`/`:RM`/`:RS` (optional but easy) [Meade Tele...d Protocol]

Defer to Phase 2

- Home/park/sleep/wake (`:h...`) [Meade Tele...d Protocol]
- Focuser (`:F...`) [Meade Tele...d Protocol]
- GPS (`:gT#`) [Meade Tele...d Protocol]
- High precision toggle (`:P#`) [Meade Tele...d Protocol]

This keeps your first iteration small while still enabling meaningful UI testing and regression tests.

5) Suggested Python data model (reference model friendly)

Use an immutable-ish snapshot + reducer style. For example:

- `TelescopeState` (dataclass):
  - `link_state`, `mount_mode`, `motion_state`
  - `current_coords`, `target_coords`
  - `slew_rate`, `precision`, `tracking_rate`
  - `busy_backoff`, `in_flight_command`, `last_error`
- `reduce(state, event) -> (new_state, effects[])`

Where `effects[]` are protocol sends, timers, or emitted UI events.

This maps cleanly to Swift later: your Swift UI observes state; actions dispatch events; effects send serial commands.

6) TDD test matrix (high value cases)

Here are concrete tests you can write immediately (they map to the spec):

1. **NAK retry**
   - Given `CONNECTED` and send `:GR#`
   - When RX is `NAK (0x15)`
   - Then state records `busy`, schedules retry with backoff, does not change coords. [Meade Tele...d Protocol]
2. **Goto happy path**
   - Set target `:Sr`, `:Sd` accepted
   - `:MS#` returns `0`
   - Transition to `SLEWING_TO_TARGET` then to `IDLE_TRACKING` when completion signal occurs (you can model completion via a simulated timer or `:D#` behavior). [Meade Tele...d Protocol]
3. **Goto rejected: below horizon**
   - `:MS#` returns `1<string>#`
   - Ensure `motion_state` returns to idle and `last_error` is `BELOW_HORIZON` with the message. [Meade Tele...d Protocol]

4 **Stop**

- From `SLEWING_TO_TARGET`, send `:Q#`
- Return to `IDLE_TRACKING`. [Meade Tele...d Protocol]

5 **Manual move serialization**

- Start `:Me#` → `MANUAL_MOVING(E)`
- While moving, attempting a goto queues or rejects (your choice, but deterministic)
- `:Qe#` returns to `IDLE_TRACKING`. [Meade Tele...d Protocol]

7) One practical recommendation: model "command-in-flight" explicitly

Because the protocol is request/response-ish with busy/NAK and timeouts, your reference model will be more realistic if it includes:

- `in_flight: Optional[Command]`
- `deadline: time`
- `retries_remaining`
- `last_nak_time`

…and a deterministic scheduler (`tick(dt)`) to drive timeouts/retries in tests.

This is where a lot of real-world LX200GPS stability comes from (NAKs and delayed responses are normal). [Meade Tele...d Protocol]

A single follow-up question (to tailor the scope without slowing you down)

For ScopeBoss' first Swift UI milestone, do you need the reference model to cover **park/home/sleep** and **focuser**, or is **connect + position + goto + stop + manual move** enough for your initial TDD suite?

If you answer that, I can turn the blueprint above into a **clean statechart diagram (textual)** and a **minimal Python skeleton** (pure reducer + effects) that you can drop directly into your test harness.

Great — adding **park/home/sleep** and **focuser** is very doable, and it *does* introduce the "unique path" you mentioned: **Park → user powers off**. There isn't a protocol-level "power off" command in the Meade set; instead, the protocol provides a **Park** command to move the mount to a safe position, and then guidance is to power off after the motors stop. [Meade Tele...d Protocol], [miltonhill.us]

Below is a concrete **scope + statechart blueprint** (with explicit transitions), plus a **Python reference-model skeleton** designed for TDD (pure reducer + effects + deterministic timers). I've grounded protocol semantics in the Meade "Telescope Serial Command Protocol" document and a practical LX200GPS mini-control page that explicitly mentions power-off after Park. [Meade Tele...d Protocol], [miltonhill.us]

1) Protocol facts to encode (directly relevant to park/home/sleep/focus)

Home/Park/Sleep/Wake

The "Home Position Commands" include:

- `:hS#` seek home and **store encoder values** at home in nonvolatile memory (on LX200GPS). [Meade Tele...d Protocol]
- `:hF#` seek home and **set/align** based on stored home encoder values. [Meade Tele...d Protocol]
- `:h?#` query home status returning: `0` failed, `1` found, `2` in progress. [Meade Tele...d Protocol]
- `:hP#` slew to **Park Position** (returns nothing). [Meade Tele...d Protocol]
- `:hN#` **Sleep** telescope: "Power off motors, encoders, displays and lights" and remain in minimum power until a keystroke or wake command. [Meade Tele...d Protocol]
- `:hW#` **Wake** sleeping telescope (LX200GPS). [Meade Tele...d Protocol]

Additionally, a practical control page states:

- "Park Telescope: Telescope goes to Park position and can then be power off. **Wait for telescope tracking motors to stop before powering off.**" [miltonhill.us]

Focuser

Focus control commands include:

- `:F+#` start focuser moving inward, `:F-#` outward, `:FQ#` halt focuser motion. [Meade Tele...d Protocol]
- Speed controls: `:FF#` fastest, `:FS#` slowest, and `:F<n>#` where `<n>` is an ASCII digit (1..4) for Autostar & LX200GPS. [Meade

Tele...d Protocol]

Busy/NAK behavior (important for TDD realism)

LX200GPS may reply **NAK (0x15)** if busy and the controller should wait and retry. [Meade Tele...d Protocol]

2) State machine scope: use orthogonal regions (statechart style)

You'll get the most stable TDD surface by modeling multiple parallel regions instead of a single giant state.

Region A — Link / Session (protocol pipe)

States

- `DISCONNECTED`
- `CONNECTING`
- `CONNECTED`
- `RECOVERING` (timeouts/NAK storms)
- `FAULTED`

Key behaviors

- Track `in_flight_command`
- Implement deterministic retry on NAK (0x15). [Meade Tele...d Protocol]

Region B — Motion / Mount Activity (includes Park/Home/Sleep)

This is the "big" region your UI and safety logic will care about.

Top-level states

- `IDLE_TRACKING`
- `SLEWING_TO_TARGET`
- `MANUAL_MOVING`
- `HOMING` (Home search/align)
- `PARKING`
- `PARKED_READY` (the "unique path" for safe power-off)
- `SLEEPING`

Home substate (HOMING)

- `HOME_SEARCH_IN_PROGRESS`
- `HOME_FOUND`
- `HOME_FAILED`

Transitions grounded by protocol

- `ANY (except SLEEPING) --(cmd :hS#)--> HOMING(HOME_SEARCH_IN_PROGRESS)` (store encoder values at home) [Meade Tele...d Protocol]
- `ANY (except SLEEPING) --(cmd :hF#)--> HOMING(HOME_SEARCH_IN_PROGRESS)` (seek home and align based on stored values) [Meade Tele...d Protocol]
- `HOMING --(poll :h?# == 2)--> HOMING(HOME_SEARCH_IN_PROGRESS)` [Meade Tele...d Protocol]
- `HOMING --(poll :h?# == 1)--> IDLE_TRACKING` (and record `home_status=FOUND`) [Meade Tele...d Protocol]
- `HOMING --(poll :h?# == 0)--> IDLE_TRACKING` (and record `home_status=FAILED`) [Meade Tele...d Protocol]

Suggestion (not in protocol text): in your *reference model*, treat `:hS#` and `:hF#` as long-running operations whose completion is determined by periodic `:h?#` polling. This matches the existence of `:h?#` and keeps TDD deterministic.

Park path (PARKING → PARKED_READY → power-off)

- `ANY (except SLEEPING) --(cmd :hP#)--> PARKING` (park command returns nothing) [Meade Tele...d Protocol]
- `PARKING --(internal "park complete" event)--> PARKED_READY`

Here's the **unique path** you asked for:

- In `PARKED_READY`, your app can allow a "Power Off Now" UI affordance. This matches the operational guidance that after Park, the telescope can be powered off, but you should wait until motors stop. [miltonhill.us]

Important: The protocol doesn't specify a park-complete response for `:hP#` (it returns nothing).
Suggestion (explicitly marked): Decide "park complete" in your reference model by either (a) a simulated timer, or (b) polling a status-like signal (some setups use other commands for motion status, but the *park command itself* doesn't provide completion info in the cited text). [Meade Tele...d Protocol]

Sleep/Wake

- `ANY --(cmd :hN#)--> SLEEPING` where sleep powers off motors/encoders/displays/lights and stays in min power until keystroke or wake command. [Meade Tele...d Protocol]
- `SLEEPING --(cmd :hW#)--> IDLE_TRACKING` (wake). [Meade Tele...d Protocol]

Suggestion: In SLEEPING, reject/queue motion & focus commands (deterministically) to make your TDD expectations clear.

Region C — Focuser (independent accessory region)

States

- `FOCUS_IDLE`
- `FOCUS_IN`
- `FOCUS_OUT`

Transitions (protocol-grounded)

- `FOCUS_IDLE --(cmd :F+#)--> FOCUS_IN` [Meade Tele...d Protocol]
- `FOCUS_IDLE --(cmd :F-#)--> FOCUS_OUT` [Meade Tele...d Protocol]
- `FOCUS_IN/FOCUS_OUT --(cmd :FQ#)--> FOCUS_IDLE` [Meade Tele...d Protocol]
- Speed setting while idle or moving:
  - `:FF#` fastest, `:FS#` slowest, `:F<n>#` (1..4) for LX200GPS. [Meade Tele...d Protocol]

Coupling rule

- If Motion region is `SLEEPING`, focuser region should be forced to `FOCUS_IDLE` (or "disabled") because sleep powers off motors/encoders/etc. [Meade Tele...d Protocol]
  - (This coupling is a **design choice**; the cited text establishes sleep powers down motors, but it doesn't explicitly mention focuser motor; implementing the safety lock is a reasonable app-level rule.)

3) "Power-off" modeling (your unique Park path)

Since your TDD needs to validate UI flows around "Park then power off":

Recommended state additions

- `PARKED_READY` (entered only after Park completes)
- `POWERED_OFF` (app-level / simulated; not a Meade command)

Why this is consistent with sources

- Park moves to a Park position via `:hP#`. [Meade Tele...d Protocol]
- Operational guidance says after Park you can power off, and you should wait for motors to stop. [miltonhill.us]

TDD-friendly approach (suggestion)

- When Park starts, schedule an internal deterministic timer (e.g., `park_duration_s`) and then transition to `PARKED_READY`.
- Your tests can assert: `park_button -> PARKING -> PARKED_READY -> user_power_off -> POWERED_OFF`.
- When "powered off", the Link region should go to `DISCONNECTED` (serial drop) and Motion to `POWERED_OFF`.

4) Python reference model skeleton (pure reducer + effects)

This is intentionally "small but complete" for the features you listed.

```
from future import annotations
from dataclasses import dataclass, replace
from enum import Enum, auto
from typing import Optional, List, Union, Tuple

# ----------------------------
# Enums / state
# ----------------------------

class LinkState(Enum):
    DISCONNECTED = auto()
    CONNECTING = auto()
    CONNECTED = auto()
    RECOVERING = auto()
    FAULTED = auto()

class MotionState(Enum):
```

```
    IDLE_TRACKING = auto()
    SLEWING_TO_TARGET = auto()
    MANUAL_MOVING = auto()
    HOMING = auto()
    PARKING = auto()
    PARKED_READY = auto()
    SLEEPING = auto()
    POWERED_OFF = auto()  # app-level / simulation

class HomeStatus(Enum):
    UNKNOWN = auto()
    IN_PROGRESS = auto()
    FOUND = auto()
    FAILED = auto()

class FocusState(Enum):
    FOCUS_IDLE = auto()
    FOCUS_IN = auto()
    FOCUS_OUT = auto()

@dataclass(frozen=True)
class InFlight:
    cmd: str
    deadline_ms: int
    retries_left: int

@dataclass(frozen=True)
class Timers:
    park_done_at_ms: Optional[int] = None
    home_poll_due_at_ms: Optional[int] = None

@dataclass(frozen=True)
class TelescopeState:
    now_ms: int = 0
    link: LinkState = LinkState.DISCONNECTED
    motion: MotionState = MotionState.POWERED_OFF
    focus: FocusState = FocusState.FOCUS_IDLE
    home_status: HomeStatus = HomeStatus.UNKNOWN
    focus_speed: Optional[int] = None  # 1..4 (or None)
    in_flight: Optional[InFlight] = None
    timers: Timers = Timers()
    last_error: Optional[str] = None

# -----------------------------
# Effects (outputs)
# -----------------------------

@dataclass(frozen=True)
class Send:
    cmd: str

@dataclass(frozen=True)
class Schedule:
    at_ms: int
    token: str  # e.g. "park_done", "home_poll", "timeout"

Effect = Union[Send, Schedule]

# -----------------------------
# Events (inputs)
# -----------------------------

@dataclass(frozen=True)
class Tick:
    dt_ms: int
```

```
@dataclass(frozen=True)
class Connect: pass

@dataclass(frozen=True)
class Disconnect: pass

@dataclass(frozen=True)
class RxNak:  # NAK 0x15
    pass

@dataclass(frozen=True)
class RxHomeQuery:
    # :h?# returns 0/1/2
    code: int

@dataclass(frozen=True)
class CmdPark: pass       # intent: park
@dataclass(frozen=True)
class CmdSleep: pass      # intent: sleep
@dataclass(frozen=True)
class CmdWake: pass       # intent: wake
@dataclass(frozen=True)
class CmdHomeStore: pass  # :hS#
@dataclass(frozen=True)
class CmdHomeAlign: pass  # :hF#

@dataclass(frozen=True)
class CmdFocusIn: pass    # :F+#
@dataclass(frozen=True)
class CmdFocusOut: pass   # :F-#
@dataclass(frozen=True)
class CmdFocusStop: pass  # :FQ#
@dataclass(frozen=True)
class CmdFocusSpeed:
    n: int                # 1..4 => :F<n>#

@dataclass(frozen=True)
class UserPowerOff: pass  # app-level event for your unique path

Event = Union[
    Tick, Connect, Disconnect, RxNak, RxHomeQuery,
    CmdPark, CmdSleep, CmdWake, CmdHomeStore, CmdHomeAlign,
    CmdFocusIn, CmdFocusOut, CmdFocusStop, CmdFocusSpeed,
    UserPowerOff,
]

# ----------------------------
# Reducer
# ----------------------------

def reduce(state: TelescopeState, event: Event) -> Tuple[TelescopeState, List[Effect]]:
    effects: List[Effect] = []
    s = state

    # time progression
    if isinstance(event, Tick):
        s = replace(s, now_ms=s.now_ms + event.dt_ms)
        # timer-based transitions (deterministic simulation)
        if s.timers.park_done_at_ms is not None and s.now_ms >= s.timers.park_done_at_ms:
            if s.motion == MotionState.PARKING:
                s = replace(s, motion=MotionState.PARKED_READY, timers=replace(s.timers, park_done_at_ms=None))
        if s.timers.home_poll_due_at_ms is not None and s.now_ms >= s.timers.home_poll_due_at_ms:
            if s.motion == MotionState.HOMING:
                # poll home status :h?#
                effects.append(Send(":h?#"))
                s = replace(s, timers=replace(s.timers, home_poll_due_at_ms=s.now_ms + 500))  # poll interval is your choice
```

```
    # in-flight timeouts
    if s.in_flight and s.now_ms >= s.in_flight.deadline_ms:
        if s.in_flight.retries_left > 0:
            effects.append(Send(s.in_flight.cmd))
            s = replace(s, in_flight=replace(s.in_flight,
                                    deadline_ms=s.now_ms + 500,
                                    retries_left=s.in_flight.retries_left - 1))
        else:
            s = replace(s, link=LinkState.RECOVERING, in_flight=None, last_error="timeout")
    return s, effects

# connect/disconnect
if isinstance(event, Connect):
    s = replace(s, link=LinkState.CONNECTING, motion=MotionState.IDLE_TRACKING)
    # handshake suggestion: query ACK alignment mode etc (not shown here)
    return s, effects

if isinstance(event, Disconnect):
    s = replace(s, link=LinkState.DISCONNECTED, in_flight=None)
    return s, effects

# NAK handling: retry with backoff (protocol says NAK 0x15 when busy) 1
if isinstance(event, RxNak):
    if s.in_flight and s.in_flight.retries_left > 0:
        effects.append(Send(s.in_flight.cmd))
        s = replace(s, in_flight=replace(s.in_flight,
                                    deadline_ms=s.now_ms + 500,
                                    retries_left=s.in_flight.retries_left - 1),
                last_error="nak_busy")
    else:
        s = replace(s, link=LinkState.RECOVERING, in_flight=None, last_error="nak_busy_exhausted")
    return s, effects

# home query response handling (:h?# returns 0/1/2) 1
if isinstance(event, RxHomeQuery):
    if event.code == 2:
        s = replace(s, home_status=HomeStatus.IN_PROGRESS)
    elif event.code == 1:
        s = replace(s, home_status=HomeStatus.FOUND, motion=MotionState.IDLE_TRACKING,
                timers=replace(s.timers, home_poll_due_at_ms=None))
    elif event.code == 0:
        s = replace(s, home_status=HomeStatus.FAILED, motion=MotionState.IDLE_TRACKING,
                timers=replace(s.timers, home_poll_due_at_ms=None))
    else:
        s = replace(s, last_error=f"invalid_home_code:{event.code}")
    return s, effects

# ----------------------------
# Motion intents: Park/Home/Sleep/Wake
# ----------------------------

if isinstance(event, CmdPark):
    if s.motion == MotionState.SLEEPING:
        return replace(s, last_error="cannot_park_while_sleeping"), effects
    # Send :hP# (park) 1
    effects.append(Send(":hP#"))
    # Park completion: app-level deterministic timer (suggestion).
    s = replace(s, motion=MotionState.PARKING,
            timers=replace(s.timers, park_done_at_ms=s.now_ms + 5000),
            last_error=None)
    return s, effects

if isinstance(event, UserPowerOff):
    # Unique path: only allowed when parked (app-level behavior)
    if s.motion == MotionState.PARKED_READY:
        s = replace(s, motion=MotionState.POWERED_OFF, link=LinkState.DISCONNECTED, in_flight=None)
```

```
    else:
        s = replace(s, last_error="power_off_not_allowed_unless_parked")
    return s, effects

if isinstance(event, CmdSleep):
    # Send :hN# (sleep) 1
    effects.append(Send(":hN#"))
    s = replace(s, motion=MotionState.SLEEPING, focus=FocusState.FOCUS_IDLE, last_error=None)
    return s, effects

if isinstance(event, CmdWake):
    # Send :hW# (wake) 1
    effects.append(Send(":hW#"))
    s = replace(s, motion=MotionState.IDLE_TRACKING, last_error=None)
    return s, effects

if isinstance(event, CmdHomeStore):
    # :hS# store home encoder values 1
    effects.append(Send(":hS#"))
    s = replace(s, motion=MotionState.HOMING, home_status=HomeStatus.IN_PROGRESS,
            timers=replace(s.timers, home_poll_due_at_ms=s.now_ms + 250))
    return s, effects

if isinstance(event, CmdHomeAlign):
    # :hF# align using stored home encoder values 1
    effects.append(Send(":hF#"))
    s = replace(s, motion=MotionState.HOMING, home_status=HomeStatus.IN_PROGRESS,
            timers=replace(s.timers, home_poll_due_at_ms=s.now_ms + 250))
    return s, effects

# ----------------------------
# Focuser intents
# ----------------------------

if isinstance(event, CmdFocusSpeed):
    # :F<n># 1..4 for LX200GPS 1
    n = event.n
    if n < 1 or n > 4:
        return replace(s, last_error="invalid_focus_speed"), effects
    if s.motion == MotionState.SLEEPING:
        return replace(s, last_error="cannot_change_focus_speed_while_sleeping"), effects
    effects.append(Send(f":F{n}#"))
    s = replace(s, focus_speed=n, last_error=None)
    return s, effects

if isinstance(event, CmdFocusIn):
    if s.motion == MotionState.SLEEPING:
        return replace(s, last_error="cannot_focus_while_sleeping"), effects
    effects.append(Send(":F+#"))  # 1
    s = replace(s, focus=FocusState.FOCUS_IN, last_error=None)
    return s, effects

if isinstance(event, CmdFocusOut):
    if s.motion == MotionState.SLEEPING:
        return replace(s, last_error="cannot_focus_while_sleeping"), effects
    effects.append(Send(":F-#"))  # 1
    s = replace(s, focus=FocusState.FOCUS_OUT, last_error=None)
    return s, effects

if isinstance(event, CmdFocusStop):
    effects.append(Send(":FQ#"))  # 1
    s = replace(s, focus=FocusState.FOCUS_IDLE, last_error=None)
    return s, effects

# unknown event
return replace(s, last_error="unhandled_event"), effects
```

Where the above is explicitly grounded vs app-level

- The commands and meanings for `:hS#` `:hF#` `:h?#` `:hP#` `:hN#` `:hW#` and focuser commands `:F+` `:F-` `:FQ` `:F<n>` come straight from the protocol document. [Meade Tele...d Protocol]
- The "Park then power off" concept is supported by the mini-control guidance ("can then be power off … wait for motors to stop"). [miltonhill.us]
- The *timer-based park completion* is a **deliberate app-side simulation choice** because the cited protocol text for `:hP#` says it returns nothing and doesn't describe completion signaling. [Meade Tele...d Protocol]

5) TDD scenarios to add immediately (park/home/sleep/focus)

1 **Home store success path**
- Dispatch `CmdHomeStore` → expect Send(`:hS#`) and `motion=HOMING`. [Meade Tele...d Protocol]
- On scheduled poll(s), model RxHomeQuery(2) then RxHomeQuery(1) → expect `home_status=FOUND` and `motion=IDLE_TRACKING`. [Meade Tele...d Protocol]

2 **Home failure path**
- RxHomeQuery(0) → `home_status=FAILED` and `motion=IDLE_TRACKING`. [Meade Tele...d Protocol]

3 **Park → ready to power off**
- Dispatch `CmdPark` → Send(`:hP#`), `motion=PARKING`. [Meade Tele...d Protocol]
- After deterministic timer tick → `motion=PARKED_READY`
- Dispatch `UserPowerOff` → `motion=POWERED_OFF` and link disconnected
- Assert UI shows "safe to power off" only in `PARKED_READY` (matches guidance). [miltonhill.us]

4 **Sleep disables movement and focuser**
- `CmdSleep` → Send(`:hN#`), `motion=SLEEPING`. [Meade Tele...d Protocol]
- While sleeping: `CmdFocusIn` rejected (your model records error); `CmdWake` → Send(`:hW#`) and `motion=IDLE_TRACKING`. [Meade Tele...d Protocol]

5 **Focuser move and stop**
- `CmdFocusSpeed(3)` → Send(`:F3#`) [Meade Tele...d Protocol]
- `CmdFocusIn` → Send(`:F+#`) and `focus=FOCUS_IN` [Meade Tele...d Protocol]
- `CmdFocusStop` → Send(`:FQ#`) and `focus=FOCUS_IDLE` [Meade Tele...d Protocol]

6) What I'd implement next (still within "scoping", but very practical)

To make the model "feel" like a real LX200GPS in tests:

- Add a **command parser** layer that maps incoming bytes to events (`RxNak`, `RxHomeQuery`, etc.). The NAK behavior is explicitly defined (0x15). [Meade Tele...d Protocol]
- Add an **operation lock policy**: e.g., while `HOMING` or `PARKING`, you might disallow `SLEWING_TO_TARGET` (your choice), but keep it deterministic.
- Add a small **capability matrix** (because the protocol doc warns some commands are accepted but may do nothing depending on implementation). [Meade Tele...d Protocol]
  - In TDD, this lets you test "feature supported" vs "no-op" cleanly.

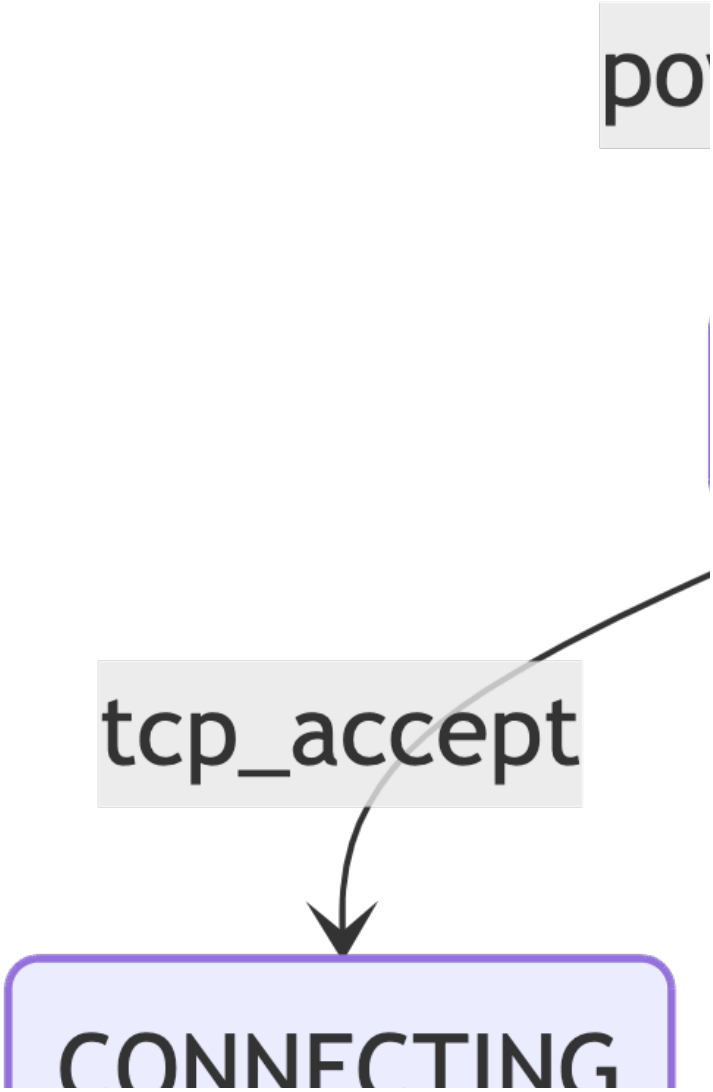
If you want, I can also provide a **textual statechart diagram** (Mermaid `stateDiagram-v2`) that mirrors the regions above, so you can paste it into docs and keep your Swift refactor aligned with the Python reference model.

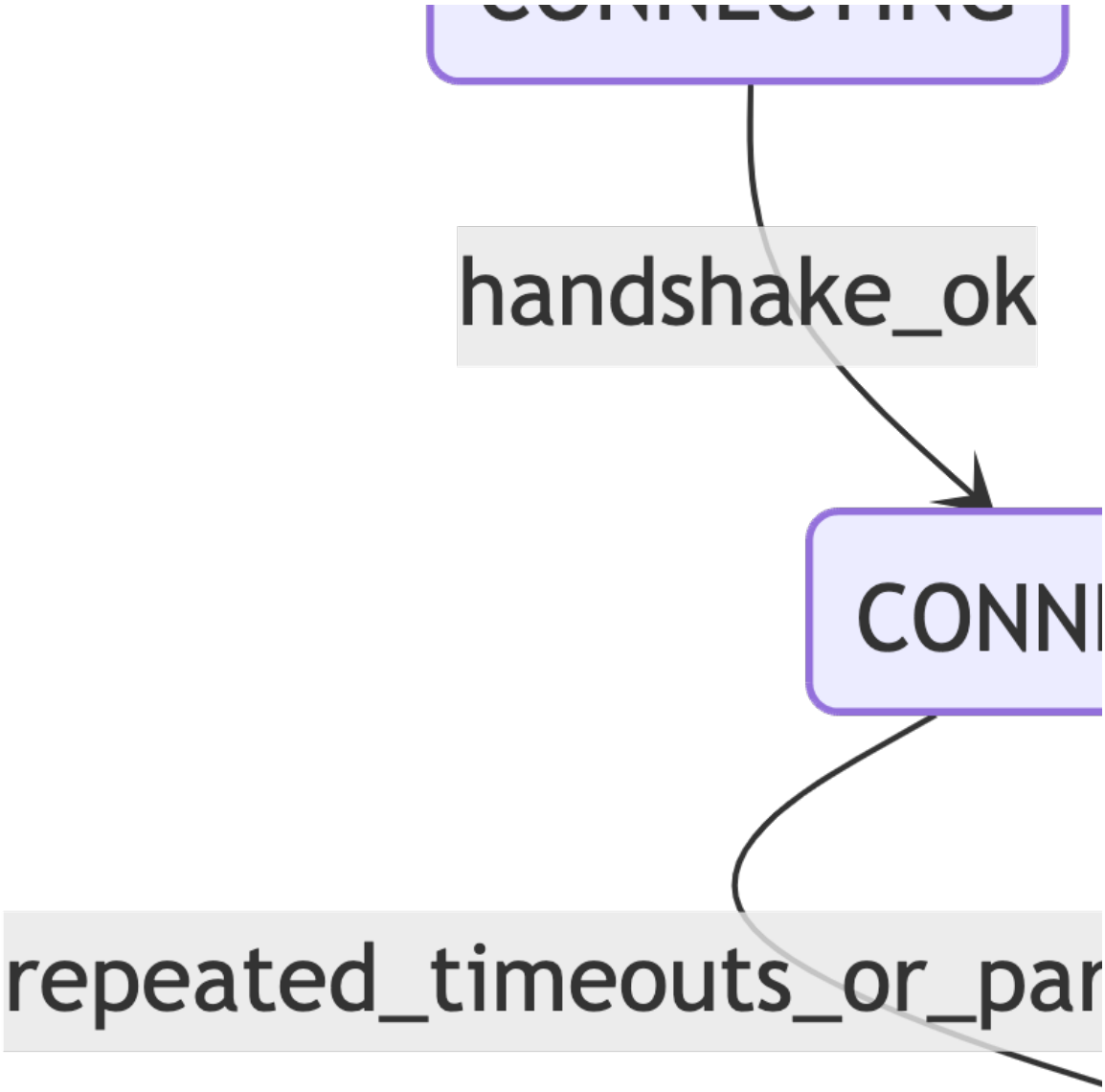
Absolutely — here's a **textual statechart (Mermaid)** plus a **TCP "serial-bus mimic" command parser** and a **deterministic operation-lock policy** tailored to **park/home/sleep + focuser**, grounded in the Meade LX200GPS protocol behaviors.

1) Textual statechart diagrams (Mermaid)

Because Mermaid doesn't do true "orthogonal regions" (parallel states) cleanly, I'm giving you **three coordinated diagrams** (Link/Power, Motion/Home/Park/Sleep, Focuser) that you run in parallel in your emulator. The **operation lock policy** (Section 3) defines how they interact.

1A. Power + Link region

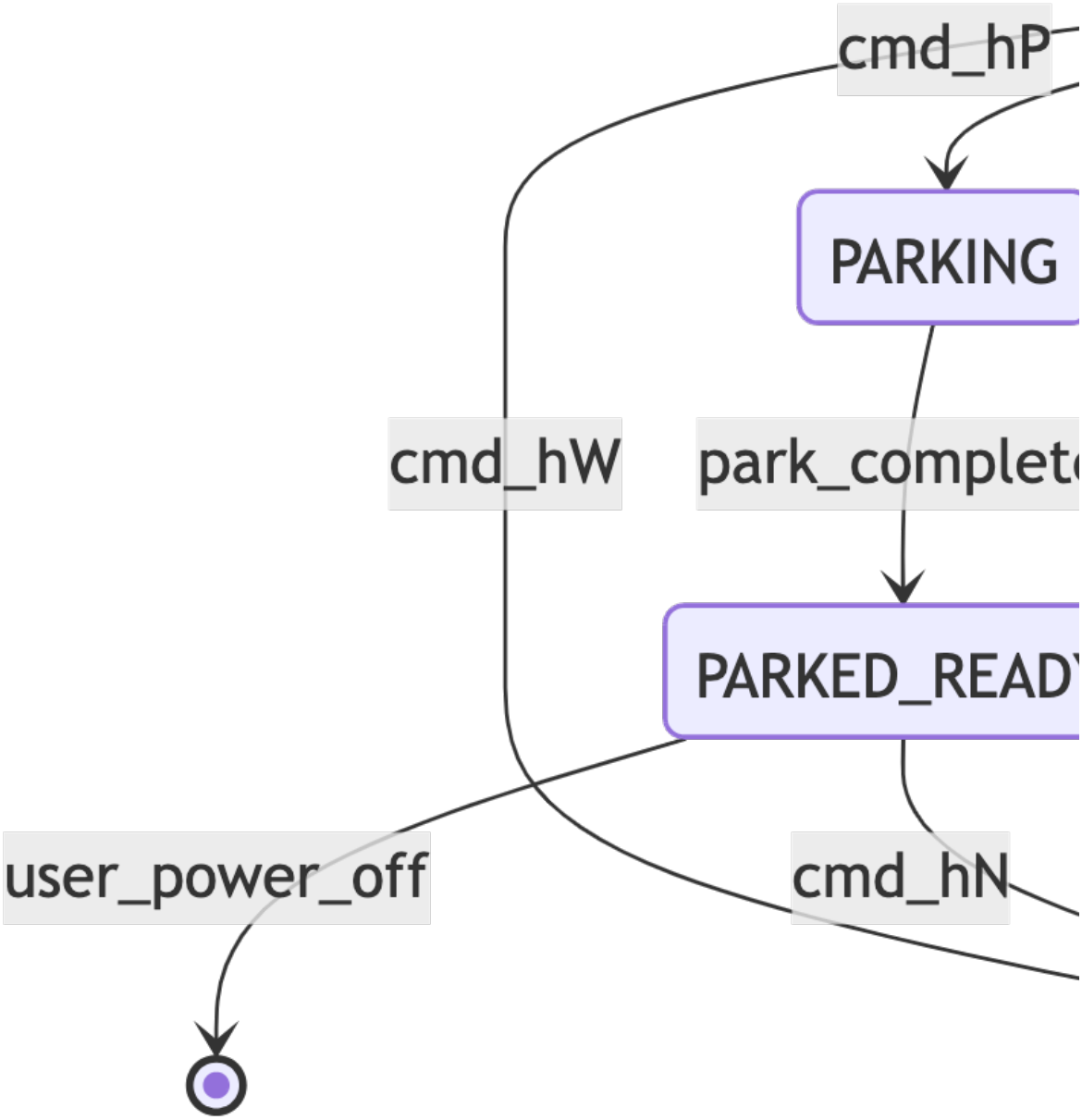

po
tcp_accept
CONNECTING



Notes

- "Handshake_ok" can be as simple as: client sends ACK (0x06) and you respond with the mount mode code. The protocol defines ACK as "alignment mounting mode query" returning mount mode indicators (AltAz/Land/Polar). [Meade Tele...d Protocol]
- In your emulator, "power_off" is app-level (no Meade command), but it's needed for the **Park → power off** TDD path (see Motion region + policy).

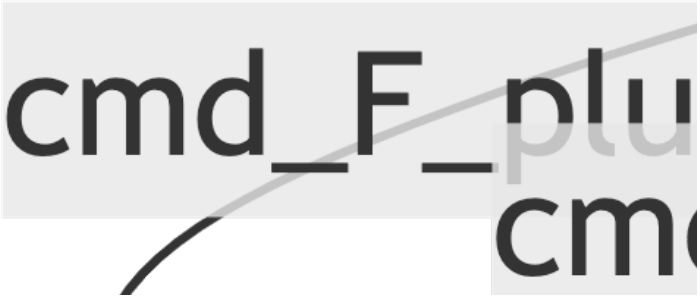
1B. Motion region (includes Home/Park/Sleep)

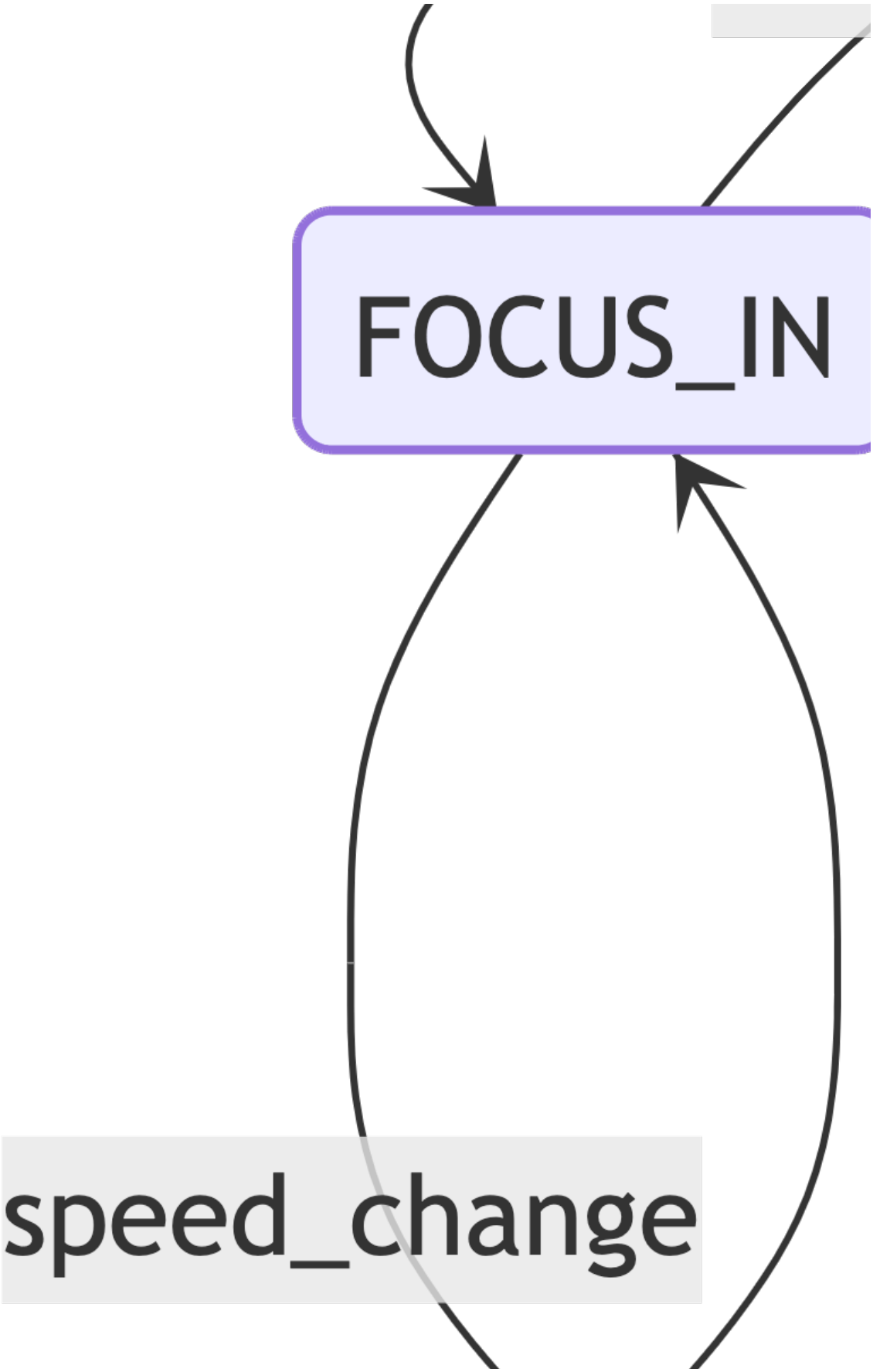

cmd_hP
PARKING
cmd_hW
park_complete
PARKED_READY
user_power_off
cmd_hN

Grounding / semantics

- `:hS#` "seeks Home Position and stores encoder values … in nonvolatile memory" (LX200GPS). [Meade Tele...d Protocol]
- `:hF#` "seeks the Home Position … and sets/aligns … based on encoder values stored in non-volatile memory". [Meade Tele...d Protocol]
- `:h?#` "Query Home Status" returns `0` failed, `1` found, `2` in progress. [Meade Tele...d Protocol]
- `:hP#` "Slew to Park Position" (returns nothing). [Meade Tele...d Protocol]
- `:hN#` "Sleep Telescope. Power off motors, encoders, displays and lights … until a keystroke is received or a wake command is sent." [Meade Tele...d Protocol]
- `:hW#` "Wake up sleeping telescope." [Meade Tele...d Protocol]
- The "unique path" **Park → power off** is operationally recommended: after Park, "can then be power off" and you should "wait for telescope tracking motors to stop before powering off."
    - Since `:hP#` itself returns nothing and does not define "park complete" signaling in the cited text, your emulator should model completion deterministically (timer or an internal completion event). [Meade Tele...d Protocol]

1C. Focuser region (parallel to motion)




FOCUS_IN
speed_change

Grounding

- `:F+#` start focuser inward, `:F-#` outward, `:FQ#` halt focuser motion. [Meade Tele...d Protocol]
- Speed: `:F<n>#` where `<n>` is ASCII digit 1..4 for Autostar & LX200GPS. [Meade Tele...d Protocol]

2) TCP "serial bus mimic" command parser (streaming)

Your emulator should behave like a **byte-stream device**: no line discipline, no JSON, no framing beyond what Meade uses (# terminator, and a few special single-byte commands).

2A. What to recognize

1 **Single-byte commands**
- `0x06` (ACK): alignment/mounting mode query. [Meade Tele...d Protocol]
- `0x15` (NAK): used by telescope as "busy" response. (In emulator, you *send* it; client won't normally send it.) [Meade Tele...d Protocol]

2 **Hash-terminated ASCII command frames**
- Most commands are ASCII and terminate with `#`. The protocol text repeatedly describes commands and responses as #-terminated strings. [Meade Tele...d Protocol]

2B. Streaming parser algorithm (robust over TCP packetization)

State

- `rx_buffer: bytearray`

On data received

- For each byte:
  - If byte == `0x06` and `rx_buffer` is empty → emit `CMD_ACK`.
  - Else append to `rx_buffer`.
  - If byte == # → decode `rx_buffer` as ASCII (strip trailing #) → emit `CMD_FRAME(text)` → clear buffer.

Optional resilience

- Ignore `\r` / `\n` if you want telnet testing; not required for correctness.
- If buffer exceeds a safety length (e.g., 256), treat as parse error → clear buffer and enter `RECOVERING`.

2C. Dispatch mapping (commands you asked to include)

When you parse a command frame string `cmd`:

Incoming	Meaning in emulator	Response
`:hP#`	Park	no response (command returns nothing) [Meade Tele...d Protocol]
`:hN#`	Sleep	no response [Meade Tele...d Protocol]
`:hW#`	Wake	no response [Meade Tele...d Protocol]
`:hS#`	Home store	no response [Meade Tele...d Protocol]
`:hF#`	Home align	no response [Meade Tele...d Protocol]
`:h?#`	Home status query	send ASCII digit `0/1/2` (failed/found/in-progress) [Meade Tele...d Protocol]
`:F+#`	Focus in	no response [Meade Tele...d Protocol]
`:F-#`	Focus out	no response [Meade Tele...d Protocol]
`:FQ#`	Focus stop	no response [Meade Tele...d Protocol]
`:F1#..:F4#`	Focus speed set	no response [Meade Tele...d Protocol]

ACK (0x06):

- Respond with the current mount mode indicator. The protocol defines ACK as "Query of alignment mounting mode" returning different codes for mount modes (AltAz/Land/Polar). [Meade Tele...d Protocol]

Implementation note (explicitly a suggestion): If you want strict realism, make `:h?#` response formatting match your real mount (some implementations may include a # terminator, others may not). The cited text specifies the numeric result codes but doesn't state the terminator for `:h?#`. [Meade Tele...d Protocol]

3) Operation lock policy (deterministic + matches “busy/NAK” behavior)

The LX200GPS protocol explicitly allows the telescope to respond **NAK (0x15)** when it is busy and cannot accept/process a command, and the controller should wait and retry. [Meade Tele...d Protocol]

That maps perfectly to a clean emulator lock policy.

3A. Define “operation locks” (what makes the emulator “busy”)

Treat these as *long-running* operations that hold a lock:

- `PARKING` started by `:hP#` [Meade Tele...d Protocol]
- `HOMING` started by `:hS#` or `:hF#` [Meade Tele...d Protocol]
- (Optionally) `SLEWING_TO_TARGET` if you later add `:MS#` goto semantics.

3B. Allowed vs NAK’d commands by motion state

Below is a deterministic policy that keeps TDD simple and is consistent with the protocol’s existence of NAK-as-busy. [Meade Tele...d Protocol]

Always allowed (even when busy):

- `:h?#` (home status query) — you need this to observe home progress. [Meade Tele...d Protocol]
- `:FQ#` (stop focusing) — safe “emergency stop” for focuser. [Meade Tele...d Protocol]
- `:hN#` (sleep) — allows immediate safe shutdown behavior. [Meade Tele...d Protocol]

Allowed only when NOT sleeping:

- `:F+#`, `:F-#`, `:F1#..:F4#` (focuser movement/speed) [Meade Tele...d Protocol]
- `:hP#`, `:hS#`, `:hF#` (park/home actions) [Meade Tele...d Protocol]

In `SLEEPING`:

- Only accept `:hW#` (wake) and optionally ACK query; everything else returns NAK (busy/unavailable). `:hW#` is explicitly the wake command for sleep. [Meade Tele...d Protocol]

In `PARKED_READY`:

- Accept `user_power_off` (app-level), and you may continue to accept `:hN#` sleep.
- Optionally NAK motion/focus commands to force the “Park → power off” safe flow (this is an app design choice, not a protocol requirement). The operational guidance is specifically about powering off after parking.

3C. Busy rule: respond with NAK quickly

When a command is disallowed by the above rules:

- Respond with a single byte `0x15` (NAK). The protocol states NAK is a possible response when the chain is busy and unable to accept/process the command. [Meade Tele...d Protocol]

Suggestion: Add a small “NAK probability” knob if you want to stress-test client retry logic. But for TDD determinism, default it to “NAK only when locked”.

4) Emulator runtime structure (TCP server + state machine)

Here’s the recommended architecture:

4A. Core loop

1. TCP server accepts **one client** (simplest, deterministic).
2. Receive bytes → **stream parser** emits command events (`ACK`, `FRAME(:hP#)`, etc.).
3. For each event:
   - Check lock policy:
     - If disallowed → send `NAK (0x15)` [Meade Tele...d Protocol]
     - Else dispatch to handler/reducer → update state, schedule timers, maybe send response bytes.
4. A periodic tick (e.g., every 50ms) advances timers:
   - If `PARKING` timer elapsed → `PARKED_READY`
   - If `HOMING` in progress → optionally auto-advance home status or require explicit `:h?#` polls (your choice; both are deterministic)

4B. Deterministic completion modeling (Park/Home)

Because:

- `:hP#` returns nothing and the cited protocol text doesn’t define a completion response, [Meade Tele...d Protocol]
- Home has an explicit status query `:h?#` with progress codes, [Meade Tele...d Protocol]

Recommended deterministic model

- **Park**
  - On `:hP#`, set `motion=PARKING`, start a fixed timer (e.g., `park_ms = 5000`), then transition to `PARKED_READY`.
  - This supports your "Park requires power-off" TDD: only allow `UserPowerOff` when `PARKED_READY`. Operationally, park is what enables safe power-off.
- **Home**
  - On `:hS#` or `:hF#`, set `motion=HOMING` and `home_status=IN_PROGRESS`.
  - `:h?#` replies `2` while in progress, then `1` when found or `0` when failed. [Meade Tele...d Protocol]
  - You can decide success/failure deterministically via config (e.g., "always succeed" default, or a scripted sequence for tests).

5) Minimal "command handlers" (what state changes to apply)

Below is a crisp behavior spec for your emulator's internal state transitions:

- `ACK (0x06)`:
  - Reply with the configured mount mode indicator (AltAz/Land/Polar). [Meade Tele...d Protocol]
- `:hN#` (sleep):
  - `motion=SLEEPING`, `focus=FOCUS_IDLE` (safety), clear any active operations; no response. Sleep powers off motors/encoders/displays/lights until wake/keystroke. [Meade Tele...d Protocol]
- `:hW#` (wake):
  - `motion=IDLE_TRACKING`, no response. [Meade Tele...d Protocol]
- `:hP#` (park):
  - `motion=PARKING`, start deterministic timer, no response. [Meade Tele...d Protocol]
  - On timer: `motion=PARKED_READY` (then allow app-level power off).
- `UserPowerOff` (app-level):
  - Only valid in `PARKED_READY`; then set `POWERED_OFF` and drop TCP connection (simulate power loss). This matches the operational flow described for park → power off.
- `:hS#` / `:hF#`:
  - `motion=HOMING`, `home_status=IN_PROGRESS`, no response. [Meade Tele...d Protocol]
- `:h?#`:
  - Reply with `0/1/2` based on current home status (`FAILED/FOUND/IN_PROGRESS`). [Meade Tele...d Protocol]
- `:F+#` / `:F-#` / `:FQ#` / `:F<n>#`:
  - Update focuser region accordingly; no response for these commands per protocol listing. [Meade Tele...d Protocol]

6) One small but important suggestion for your Swift/TDD integration

Since you're running the emulator as a separate TCP program:

- Keep the TCP stream **byte-identical** to serial (including returning raw `0x15` NAK and `0x06` ACK responses). This lets your iOS transport layer swap "serial" vs "tcp" with minimal branching and keeps TDD faithful to how the real mount behaves under busy conditions. [Meade Tele...d Protocol]

If you want, I can also provide a **ready-to-run minimal Python TCP server skeleton** (asyncio) that implements:

- streaming parser (# + ACK),
- the lock policy (NAK on disallowed),
- deterministic timers for park/home,
- and the statechart variables above.

(That would still be "text + code snippet" in-chat, unless you explicitly want me to generate a file.)